Python Operators

Python Operators

There are seven kinds of operators in Python. They are

- Arithmetic Operators
- Bitwise Operators
- Assignment Operators
- Comparison Operators / Relational Operators
- Identity Operators
- Membership Operators

In this tutorial, we will go through each of these operators with examples.

Arithmetic Operators

Arithmetic Operators are used to perform basic mathematical arithmetic operators like addition, subtraction, multiplication, etc. The following table lists out all the arithmetic operators in Python.

Operator Symbol	Description	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor division	x // y

Python Arithmetic Operators

In the following program, we will take values for variables x and y, and perform arithmetic operations on these values using Python Arithmetic Operators.

Python Program

```python
x = 5
y = 2

addition = x + y
subtraction = x - y
multiplication = x * y
division = x / y
modulus = x % y
exponentiation = x ** y
floor_division = x // y

print(f"x + y = {addition}")
print(f"x - y = {subtraction}")
print(f"x * y = {multiplication}")
print(f"x / y = {division}")
print(f"x % y = {modulus}")
print(f"x ** y = {exponentiation}")
print(f"x // y = {floor_division}")
```

Output

```
x + y = 7
x - y = 3
x * y = 10
x / y = 2.5
x % y = 1
x ** y = 25
x // y = 2
```

Bitwise Operators

Bitwise Operators are used to perform bit level operations. The following table lists out all the bitwise operators in Python.

Operator Symbol	Description	Example
&	AND	x & y
|	OR	x | y
^	XOR	x ^ y
~	NOT	~x
<<	Zero fill left shift	x << y
>>	Signed right shift	x >> y

Python Bitwise Operators

Python Program

```
# AND
x, y = 5, 2
print(x & y) # 0

# OR
x, y = 5, 2
print(x | y) # 7

# XOR
x, y = 5, 2
print(x ^ y) # 7

# NOT
x, y = 5, 2
print(~x) # -6

# Zero fill left shift
x, y = 5, 2
print(x << y) # 20

#Signed right shift
x, y = 5, 2
print(x >> y) # 1
```

Assignment Operators

Assignment Operators are used to assign or store a specific value in a variable. The following table lists out all the assignment operators in Python.

Operator Symbol	Description	Example	Equivalent to
=	Assignment	x = y	
+=	Addition Assignment	x += y	x = x + y
-=	Subtraction Assignment	x -= y	x = x – y
*=	Multiplication Assignment	x *= y	x = x * y
/=	Division Assignment	x /= y	x = x / y
%=	Modulus Assignment	x %= y	x = x % y
**=	Exponentiation Assignment	x **= y	x = x ** y
//=	Floor-division Assignment	x //= y	x = x // y
&=	AND Assignment	x &= y	x = x & y
|=	OR Assignment	x |= y	x = x | y
^=	XOR Assignment	x ^= y	x = x ^ y
<<=	Zero fill left shift Assignment	x <<= y	x = x << y
>>=	Signed right shift Assignment	x >>= y	x = x >> y

In the following program, we will take values for variables x and y, and perform assignment operations on these values using Python Assignment Operators.

Python Program

```
x, y = 5, 2
x += y
print(x)  # 7

x, y = 5, 2
x -= y
print(x)  # 3

x, y = 5, 2
x *= y
print(x)  # 10

x, y = 5, 2
x /= y
print(x)  # 2.5

x, y = 5, 2
x %= y
print(x)  # 1

x, y = 5, 2
x **= y
print(x)  # 25

x, y = 5, 2
x //= y
print(x)  # 2

x, y = 5, 2
x &= y
print(x)  # 0

x, y = 5, 2
x |= y
print(x)  # 7

x, y = 5, 2
x ^= y
print(x)  # 7

x, y = 5, 2
x <<= y
print(x)  # 20

x, y = 5, 2
x >>= y
print(x)  # 1
```

Comparison Operators

Comparison Operators are used to compare two operands. The following table lists out all the Comparison operators in Python.

Operator Symbol	Description	Example
==	Equal to	x == y
!=	Not Equal to	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Python Comparison Operators

Python Program

```
# Equal to
x, y = 5, 2
print(x == y) # False

# Not equal to
x, y = 5, 2
print(x != y) # True

# Greater than
x, y = 5, 2
print(x > y) # True

# Less than
x, y = 5, 2
print(x < y) # False

# Greater than or equal to
x, y = 5, 2
print(x >= y) # True

# Less than or equal to
x, y = 5, 2
print(x <= y) # False
```

Logical Operators

Logical Operators are used to combine simple conditions and form compound conditions. The following table lists out all the Logical operators in Python.

Operator Symbol	Description	Example
and	Returns True if both operands are True.	x and y
or	Returns True if any of the operands is True.	x or y
not	Returns the complement of given boolean operand.	not x

Python Logical Operators

Python Program

```
# Logical AND
x, y = True, False
print(x and y) # False

# Logical OR
x, y = True, False
print(x or y) # True

# Logical NOT
x = True
print(not x) # False
```

Identity Operators

Identity Operators are used to check if two variables point to same reference of an object in Python. The following table lists out the two Identity operators in Python.

Operator Symbol	Description	Example
is	Returns True if both operands refer to same object.	x is y
is not	Returns True if two operands refer to different objects.	x is not y

Python Logical Operators

Two objects are said to have same reference, if they have same id. In the following program, we shall print the ids of x and y, along with the results of is and is not operators.

Python Program – x and y with same reference

```
x = [1, 2, 3]
y = x
print(x is y) # True
print(x is not y) # False
print(id(x))
print(id(y))
```

Output

```
True
False
2284566373000
2284566373000
```

We have assigned the value of x to y. Now, both x and y store the reference to same object in memory. Therefore, x is y.

Python Program – x and y with different reference

```python
# is operator
x = [1, 2, 3]
y = [1, 2, 3]
print(x is y) # True
print(x is not y) # False
print(id(x))
print(id(y))
```

Output

```
False
True
1961841222280
1961841222344
```

Even though the list elements are same, x and y are assigned with two different list objects. Hence, the ids are different for x and y and therefore x is not y, in this case.

Membership Operators

Membership Operators are used to check if an element or item is present in the given collection or sequence. The following table lists out the two Membership operators in Python.

Operator Symbol	Description	Example
in	Returns True if element (x) is present in sequence (y).	x in y
not in	Returns True if element (x) is not present in sequence (y).	x not in y

Python Logical Operators

Python Program

```python
# in operator
x = 1
y = [1, 2, 3]
print(x in y) # True

# not in operator
x = 8
y = [1, 2, 3]
print(x not in y) # True
```

Python Conditional Statements

Conditional Statements are features of a **programming** language, which perform different computations or actions depending on whether the given **condition** evaluates to true or false. Conditional statements in python are of 3 types

- **If statement**
- **If else statement**
- **If elif statement**
- **Nested if else**

1. If Statement : if Statement is used to run a statement conditionally i.e. if given condition is true then only the statement given in if block will be executed.

```
if <condition>:
    <if statement block >

For example consider the code given below
if (percentage > 33):
        print ("Pass")

Explaination : In the above code if value of percentage is above 33 then only
the message "Pass" will be printed.
```

2. If else Statement In the case of if else statement If given condition is true then the statement given in if block will be executed otherwise(else) the statements written in else block will be executed

```
if <condition>:
    <if statement block >
else:
    <else statement block>
For example consider the code given below
if (percentage > 33):
        print ("Pass")
else:
        print("Fail")
Explaination : In the above code if value of percentage is above 33 then only
the message "Pass" will be printed otherwise it will print "Fail"
```

3. If elif Statement if elif is used for execution OF STATEMENTS based on several alternatives. Here we use one or more elif (short form of *else if*) clauses. Python evaluates each condition in turn and executes the statements corresponding to the first if that is true. If none of the expressions are true, and an else clause will be executed

Syntax:-

```
if <condition>:
    <statement(s)>
elif <condition>:
    <statement(s)>
.
.
else:
    <statement(s)>
```

Explaination : In the above code if value of percentage is above 33 then only the message "Pass" will be printed otherwise it will print "Fail"

Example:

```
If (percentage >90):
        Print("Outstanding")
elif (percentage >80):
        print ("Excellent")
elif (percentage >70):
        print ("VeryGood")
elif (percentage >60):
        print ("Good")
elif (percentage >33):
        print ("Pass")
else
        print("Fail")
```

Explaination : In the above code

if value of percentage is above 90 then it will print "Outstanding"

if value of percentage is above 80 then it will print "Excellent"

if value of percentage is above 70 then it will print "Very Good"

if value of percentage is above 60 then it will print "Good"

if value of percentage is above 80 then it will print "Pass"

if no condition is true then it will print "Fail"

In above code only 1 condition can be true at a time if no condition is true then else statement will be executed

4. Nested If else Statement A nested if is an if statement that is the target of another if statement. Nested if statement means an if statement within another if statement.

Syntax:-

```
if (<condition1>):
    statement(s)
    if (<condition2>):
        statement(s)
    else
else:
    if (<condition3>):
         statement(s)
    else
        Statement(s)
```

```
# Example
if color =”red”:
   if item=”fruit”:
         print(“ It is an Apple”)
   else :
         print(“It may be Tomato or Rose”)
else:
    if  color=”Yellow”:
         print(“It is a Banana”)
    else
         print(“It may be corn or Marigold ”)
```

OR

```
if color =”red”:
   if item=”fruit”:
         print(“ It is an Apple”)
   else :
         print(“It may be Tomato or Rose”)
elif color=”Yellow”:
         print(“It is a Banana”)
    else
         print(“It may be corn or Marigold ”)
```

Python Iterative Statements

Iteration statements or loop statements allow us to execute a block of statements repeatedly as long as the condition is true.

(Loops statements are used when we need to run same code again and again)

Type of Iteration Statements In Python 3
In Python Iteration (Loops) statements are of three types :-

1. While Loop

2. For Loop

3. Nested Loops

---

1. While Loop In Python

While Loop In Python is used to execute a block of statement till the given condition is true. And when the condition is false, the control will come out of the loop.

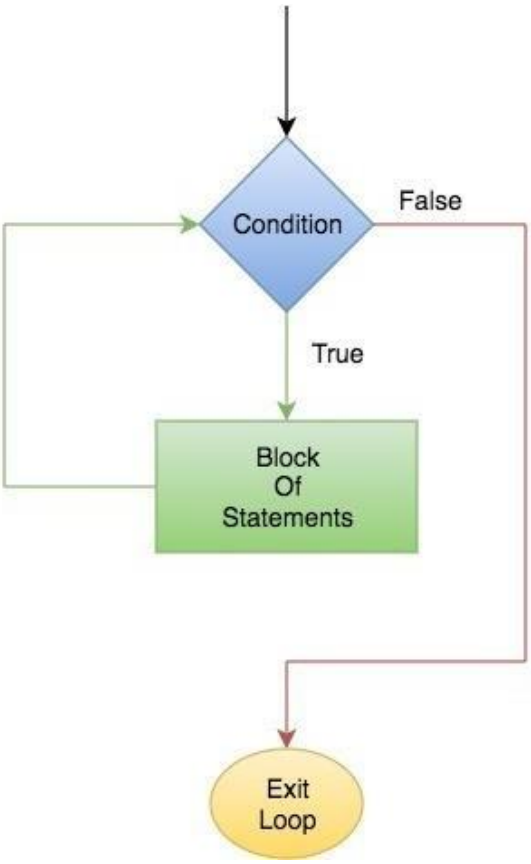
The condition is checked every time at the beginning of the loop.

While Loop Syntax

```
while (<condition>):

        statements
```

Flowchart of While Loop



Python Flowchart of While Loop

Examples Of While Loop

x = 0
while (x < 5):
  print(x)
  x = x + 1
 	
Output :-
0
1
2
3
4
	x = 1
	while (x <= 5):
	  print(—Welcome –)
	  x = x + 1
	
	Output :-
	Welcome
	Welcome
	Welcome
	Welcome
	Welcome

While Loop With Else In Python

The else part is executed if the condition in the while loop becomes False.

Syntax of While Loop With Else

```
while (condition):
        loop statements
else:
        else statements
```

Example of While Loop With Else

```
x = 1
while (x < 5):
  print(_inside while loop value of x is _,x)
  x = x + 1
else:
  print(_inside else value of x is _, x)
```

Output :-

inside while loop value of x is 1
inside while loop value of x is 2
inside while loop value of x is 3
inside while loop value of x is 4
inside else value of x is 5

****Infinite While Loop In Python** A Infinite loop is a loop in which condition always remain True.

Example of Infinite While Loop

```
x = 1
while (x == 1):
  print(_hello‘)
```

Output :-

hello
hello
hello
___
___

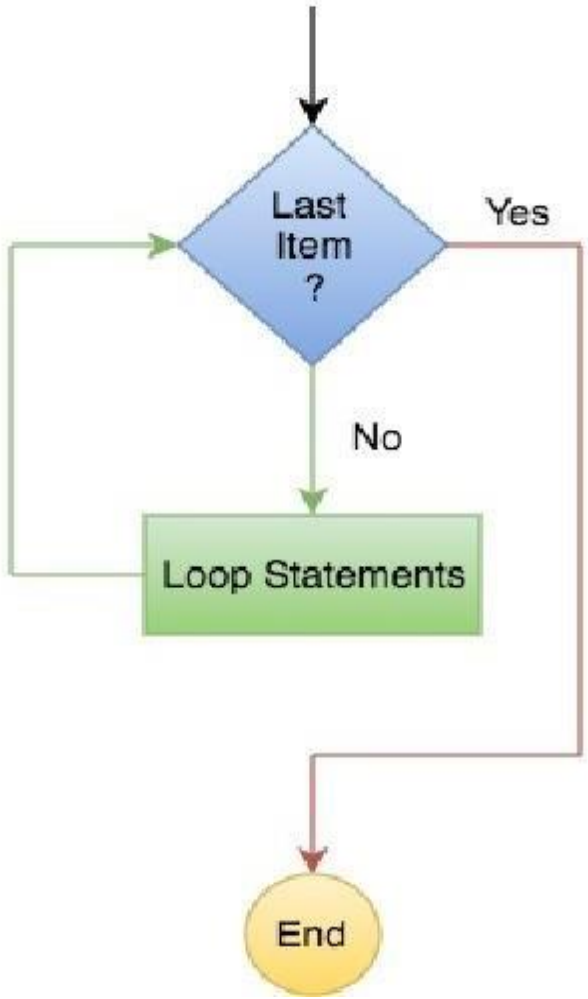
2. For Loop In Python

For loop in Python is used to iterate over items of any sequence, such as a list or a string.

For Loop Syntax

```
for val in sequence:
        statements
```

Flowchart of For Loop



Python Flowchart of For Loop

ExampleS of For Loop

for i in range(1,5): print(i) Output :- 1 2 3 4	for i in [1,2,3,4] : print(—WELCOME ‖) Output :- WELCOME WELCOME WELCOME WELCOME

****The range() Function In Python

The range() function is a built-in that is used to iterate over a sequence of numbers.

Syntax Of range() Function range(start, stop[, step])

The range() Function Parameters

start: Starting number of the sequence.

stop: Generate numbers up to, but not including this number.

step(Optional): Determines the increment between each numbers in the sequence

Example 1 of range() function	**Example 2 of range() function**
for i in range(5): print(i) **Output :-** 0 1 2 3 4	for i in range(2,9): print(i) **Output :-** 2 3 4 5 6 7 8
Example 3 of range() function using step parameter	**Example 4 of range() function**
for i in range(2,9,2): print(i) **Output :-** 2 4 6 8	for i in range(0,-10,-2): print(i) Run Code **Output :-** 0 -2 -4 -6 -8

For Loop With Else In Python

The else is an optional block that can be used with for loop.Theelse block with for loop executed only if for loops terminatesnormally.This means that the loop did not encounter any break.

Example 1 of For Loop With Else

```
list=[2,3,4,6,7]
for i in range(0,len(list)):
  if(list[i]==4):
    print(_list has 4‘)
else:
  print(_list does not have 4‘)
```

Run Code

Output :-
List has 4
list does not have 5

Example 2 of For Loop With Else

```
for i in range(0,len(list)):
  if(list[i]==5):
    print(_5 is there in the list‘)
    break
else:
  print(_list does not have 5‘)
```

Run Code

Output :-
list does not have 5

NESTED loops in Python : The placing of one **loop** inside the body of another **loop** is called **nesting**. When you "nest" two **loops**, the outer **loop** takes control of the number of complete repetitions of the inner **loop**

1.Nested while loop

Syntax :

```
initialization
while(condition):
        initialization of inner loop
        while(conition):
                ------
                ------
                Update expression of inner loop
 Update expression of outer loop
```

Examples

```
# To Print Pyramid
'''
1
1 2
1 2 3
1 2 3 4'''
i=1
while(i<=4):
    j=1
    while(j<=i):
        print(j,end=' ')
        j+=1
    print("")
    i+=1

# To Print Pyramid
'''
5
5 4
5 4 3
5 4 3 2
5 4 3 2 1'''
i=5
while(i>=1):
    j=5
    while(j>=i):
        print(j,end=' ')
        j-=1
    print("")
    i-=1
```

1. Nested for loop

Syntax

```
for iterating_var in sequence:
    for iterating_var in sequence:
        statements(s)
    statements(s)
```

Example

```
# To Print Pyramid
'''
1
1 2
1 2 3
1 2 3 4'''

for i in range(1,5):
    for j in range(1,i+1):
        print(j,end=' ')
    print("")


# To Print Pyramid
'''
5
5 4
5 4 3
5 4 3 2
5 4 3 2 1'''

for i in range(5,0,-1):
    for j in range(5,i-1,-1):
        print(j,end=' ')
    print("")
```

PYTHON *BREAK, CONTINUE AND PASS* STATEMENTS

 Copyright © tutorialspoint.com

You might face a situation in which you need to exit a loop completely when an external condition is triggered or there may also be a situation when you want to skip a part of the loop and start next execution.

Python provides **break** and **continue** statements to handle such situations and to have good control on your loop.

This tutorial will discuss the *break*, *continue* and *pass* statements available in Python.

THE *BREAK* STATEMENT:

The **break** statement in Python terminates the current loop and resumes execution at the next statement, just like the traditional break found in C.

The most common use for break is when some external condition is triggered requiring a hasty exit from a loop. The **break** statement can be used in both *while* and *for* loops.

Example:

```
#!/usr/bin/python

for letter in 'Python':     # First Example
   if letter == 'h':
      break
   print 'Current Letter :', letter

var = 10                    # Second Example
while var > 0:
   print 'Current variable value :', var
   var = var -1
   if var == 5:
      break

print "Good bye!"
```

This will produce the following result:

```
Current Letter : P
Current Letter : y
Current Letter : t
Current variable value : 10
Current variable value : 9
Current variable value : 8
Current variable value : 7
Current variable value : 6
Good bye!
```

THE *CONTINUE* STATEMENT:

The **continue** statement in Python returns the control to the beginning of the while loop. The **continue** statement rejects all the remaining statements in the current iteration of the loop and moves the control back to the top of the loop.

The **continue** statement can be used in both *while* and *for* loops.

Example:

```
#!/usr/bin/python

for letter in 'Python':     # First Example
```

```
   if letter == 'h':
      continue
   print 'Current Letter :', letter

var = 10                    # Second Example
while var > 0:
   var = var -1
   if var == 5:
      continue
   print 'Current variable value :', var
print "Good bye!"
```

This will produce following result:

```
Current Letter : P
Current Letter : y
Current Letter : t
Current Letter : o
Current Letter : n
Current variable value : 10
Current variable value : 9
Current variable value : 8
Current variable value : 7
Current variable value : 6
Current variable value : 4
Current variable value : 3
Current variable value : 2
Current variable value : 1
Good bye!
```

THE *ELSE* STATEMENT USED WITH LOOPS

Python supports to have an **else** statement associated with a loop statements.

- If the **else** statement is used with a **for** loop, the **else** statement is executed when the loop has exhausted iterating the list.
- If the **else** statement is used with a **while** loop, the **else** statement is executed when the condition becomes false.

Example:

The following example illustrates the combination of an else statement with a for statement that searches for prime numbers from 10 through 20.

```
#!/usr/bin/python

for num in range(10,20):  #to iterate between 10 to 20
   for i in range(2,num): #to iterate on the factors of the number
      if num%i == 0:      #to determine the first factor
         j=num/i #to calculate the second factor
         print '%d equals %d * %d' % (num,i,j)
         break #to move to the next number, the #first FOR
   else:        # else part of the loop
      print num, 'is a prime number'
```

This will produce following result:

```
10 equals 2 * 5
11 is a prime number
12 equals 2 * 6
13 is a prime number
14 equals 2 * 7
15 equals 3 * 5
16 equals 2 * 8
17 is a prime number
18 equals 2 * 9
```

```
19 is a prime number
```

Similar way you can use **else** statement with **while** loop.

THE *PASS* STATEMENT:

The **pass** statement in Python is used when a statement is required syntactically but you do not want any command or code to execute.

The **pass** statement is a *null* operation; nothing happens when it executes. The **pass** is also useful in places where your code will eventually go, but has not been written yet $e.g.$, $instubsforexample$:

Example:

```
#!/usr/bin/python

for letter in 'Python':
   if letter == 'h':
      pass
      print 'This is pass block'
   print 'Current Letter :', letter

print "Good bye!"
```

This will produce following result:

```
Current Letter : P
Current Letter : y
Current Letter : t
This is pass block
Current Letter : h
Current Letter : o
Current Letter : n
Good bye!
```

The preceding code does not execute any statement or code if the value of *letter* is 'h'. The *pass* statement is helpful when you have created a code block but it is no longer required.

You can then remove the statements inside the block but let the block remain with a pass statement so that it doesn't interfere with other parts of the code.

Loading [MathJax]/jax/output/HTML-CSS/fonts/TeX/fontdata.js

PYTHON FUNCTIONS

 Copyright © tutorialspoint.com

A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing.

As you already know, Python gives you many built-in functions like print, etc. but you can also create your own functions. These functions are called *user-defined functions*.

Defining a Function

You can define functions to provide the required functionality. Here are simple rules to define a function in Python.

- Function blocks begin with the keyword **def** followed by the function name and parentheses ( ).
- Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.
- The first statement of a function can be an optional statement - the documentation string of the function or *docstring*.
- The code block within every function starts with a colon : and is indented.
- The statement return [expression] exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as return None.

Syntax

```
def functionname( parameters ):
   "function_docstring"
   function_suite
   return [expression]
```

By default, parameters have a positional behavior and you need to inform them in the same order that they were defined.

Example

The following function takes a string as input parameter and prints it on standard screen.

```
def printme( str ):
   "This prints a passed string into this function"
   print str
   return
```

Calling a Function

Defining a function only gives it a name, specifies the parameters that are to be included in the function and structures the blocks of code.

Once the basic structure of a function is finalized, you can execute it by calling it from another function or directly from the Python prompt. Following is the example to call printme function –

```
#!/usr/bin/python

# Function definition is here
def printme( str ):
   "This prints a passed string into this function"
   print str
   return;
```

```
# Now you can call printme function
printme("I'm first call to user defined function!")
printme("Again second call to the same function")
```

When the above code is executed, it produces the following result –

```
I'm first call to user defined function!
Again second call to the same function
```

Pass by reference vs value

All parameters *arguments* in the Python language are passed by reference. It means if you change what a parameter refers to within a function, the change also reflects back in the calling function. For example –

```
#!/usr/bin/python

# Function definition is here
def changeme( mylist ):
   "This changes a passed list into this function"
   mylist.append([1,2,3,4]);
   print "Values inside the function: ", mylist
   return

# Now you can call changeme function
mylist = [10,20,30];
changeme( mylist );
print "Values outside the function: ", mylist
```

Here, we are maintaining reference of the passed object and appending values in the same object . So, this would produce the following result –

```
Values inside the function:  [10, 20, 30, [1, 2, 3, 4]]
Values outside the function:  [10, 20, 30, [1, 2, 3, 4]]
```

There is one more example where argument is being passed by reference and the reference is being overwritten inside the called function.

```
#!/usr/bin/python

# Function definition is here
def changeme( mylist ):
   "This changes a passed list into this function"
   mylist = [1,2,3,4]; # This would assig new reference in mylist
   print "Values inside the function: ", mylist
   return

# Now you can call changeme function
mylist = [10,20,30];
changeme( mylist );
print "Values outside the function: ", mylist
```

The parameter *mylist* is local to the function changeme. Changing mylist within the function does not affect *mylist*. The function accomplishes nothing and finally this would produce the following result:

```
Values inside the function:  [1, 2, 3, 4]
Values outside the function:  [10, 20, 30]
```

Function Arguments

You can call a function by using the following types of formal arguments:

- Required arguments

- Keyword arguments
- Default arguments
- Variable-length arguments

Required arguments

Required arguments are the arguments passed to a function in correct positional order. Here, the number of arguments in the function call should match exactly with the function definition.

To call the function *printme*, you definitely need to pass one argument, otherwise it gives a syntax error as follows –

```
#!/usr/bin/python

# Function definition is here
def printme( str ):
   "This prints a passed string into this function"
   print str
   return;

# Now you can call printme function
printme()
```

When the above code is executed, it produces the following result:

```
Traceback (most recent call last):
  File "test.py", line 11, in <module>
    printme();
TypeError: printme() takes exactly 1 argument (0 given)
```

Keyword arguments

Keyword arguments are related to the function calls. When you use keyword arguments in a function call, the caller identifies the arguments by the parameter name.

This allows you to skip arguments or place them out of order because the Python interpreter is able to use the keywords provided to match the values with parameters. You can also make keyword calls to the *printme* function in the following ways –

```
#!/usr/bin/python

# Function definition is here
def printme( str ):
   "This prints a passed string into this function"
   print str
   return;

# Now you can call printme function
printme( str = "My string")
```

When the above code is executed, it produces the following result –

```
My string
```

The following example gives more clear picture. Note that the order of parameters does not matter.

```
#!/usr/bin/python

# Function definition is here
def printinfo( name, age ):
   "This prints a passed info into this function"
   print "Name: ", name
```

```
   print "Age ", age
   return;

# Now you can call printinfo function
printinfo( age=50, name="miki" )
```

When the above code is executed, it produces the following result –

```
Name:  miki
Age 50
```

Default arguments

A default argument is an argument that assumes a default value if a value is not provided in the function call for that argument. The following example gives an idea on default arguments, it prints default age if it is not passed –

```
#!/usr/bin/python

# Function definition is here
def printinfo( name, age = 35 ):
   "This prints a passed info into this function"
   print "Name: ", name
   print "Age ", age
   return;

# Now you can call printinfo function
printinfo( age=50, name="miki" )
printinfo( name="miki" )
```

When the above code is executed, it produces the following result –

```
Name:  miki
Age 50
Name:  miki
Age 35
```

Variable-length arguments

You may need to process a function for more arguments than you specified while defining the function. These arguments are called *variable-length* arguments and are not named in the function definition, unlike required and default arguments.

Syntax for a function with non-keyword variable arguments is this –

```
def functionname([formal_args,] *var_args_tuple ):
   "function_docstring"
   function_suite
   return [expression]
```

An asterisk ∗ is placed before the variable name that holds the values of all nonkeyword variable arguments. This tuple remains empty if no additional arguments are specified during the function call. Following is a simple example –

```
#!/usr/bin/python

# Function definition is here
def printinfo( arg1, *vartuple ):
   "This prints a variable passed arguments"
   print "Output is: "
   print arg1
   for var in vartuple:
      print var
   return;
```

```
# Now you can call printinfo function
printinfo( 10 )
printinfo( 70, 60, 50 )
```

When the above code is executed, it produces the following result –

```
Output is:
10
Output is:
70
60
50
```

The *Anonymous* Functions

These functions are called anonymous because they are not declared in the standard manner by using the *def* keyword. You can use the *lambda* keyword to create small anonymous functions.

- Lambda forms can take any number of arguments but return just one value in the form of an expression. They cannot contain commands or multiple expressions.
- An anonymous function cannot be a direct call to print because lambda requires an expression
- Lambda functions have their own local namespace and cannot access variables other than those in their parameter list and those in the global namespace.
- Although it appears that lambda's are a one-line version of a function, they are not equivalent to inline statements in C or C++, whose purpose is by passing function stack allocation during invocation for performance reasons.

Syntax

The syntax of *lambda* functions contains only a single statement, which is as follows –

```
lambda [arg1 [,arg2,.........argn]]:expression
```

Following is the example to show how *lambda* form of function works –

```
#!/usr/bin/python

# Function definition is here
sum = lambda arg1, arg2: arg1 + arg2;



# Now you can call sum as a function
print "Value of total : ", sum( 10, 20 )
print "Value of total : ", sum( 20, 20 )
```

When the above code is executed, it produces the following result –

```
Value of total :  30
Value of total :  40
```

The *return* Statement

The statement return [expression] exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as return None.

All the above examples are not returning any value. You can return a value from a function as follows –

```
#!/usr/bin/python
```

```
# Function definition is here
def sum( arg1, arg2 ):
   # Add both the parameters and return them."
   total = arg1 + arg2
   print "Inside the function : ", total
   return total;

# Now you can call sum function
total = sum( 10, 20 );
print "Outside the function : ", total
```

When the above code is executed, it produces the following result –

```
Inside the function : 30
Outside the function :  30
```

Scope of Variables

All variables in a program may not be accessible at all locations in that program. This depends on where you have declared a variable.

The scope of a variable determines the portion of the program where you can access a particular identifier. There are two basic scopes of variables in Python –

- Global variables
- Local variables

Global vs. Local variables

Variables that are defined inside a function body have a local scope, and those defined outside have a global scope.

This means that local variables can be accessed only inside the function in which they are declared, whereas global variables can be accessed throughout the program body by all functions. When you call a function, the variables declared inside it are brought into scope. Following is a simple example –

```
#!/usr/bin/python

total = 0; # This is global variable.
# Function definition is here
def sum( arg1, arg2 ):
   # Add both the parameters and return them."
   total = arg1 + arg2; # Here total is local variable.
   print "Inside the function local total : ", total
   return total;

# Now you can call sum function
sum( 10, 20 );
print "Outside the function global total : ", total
```

When the above code is executed, it produces the following result –

```
Inside the function local total : 30
Outside the function global total :  0
```

Loading [MathJax]/jax/output/HTML-CSS/jax.js